

Mastering Python OOP

A Deep Dive into Advanced Operator Overloading



01. The Framework

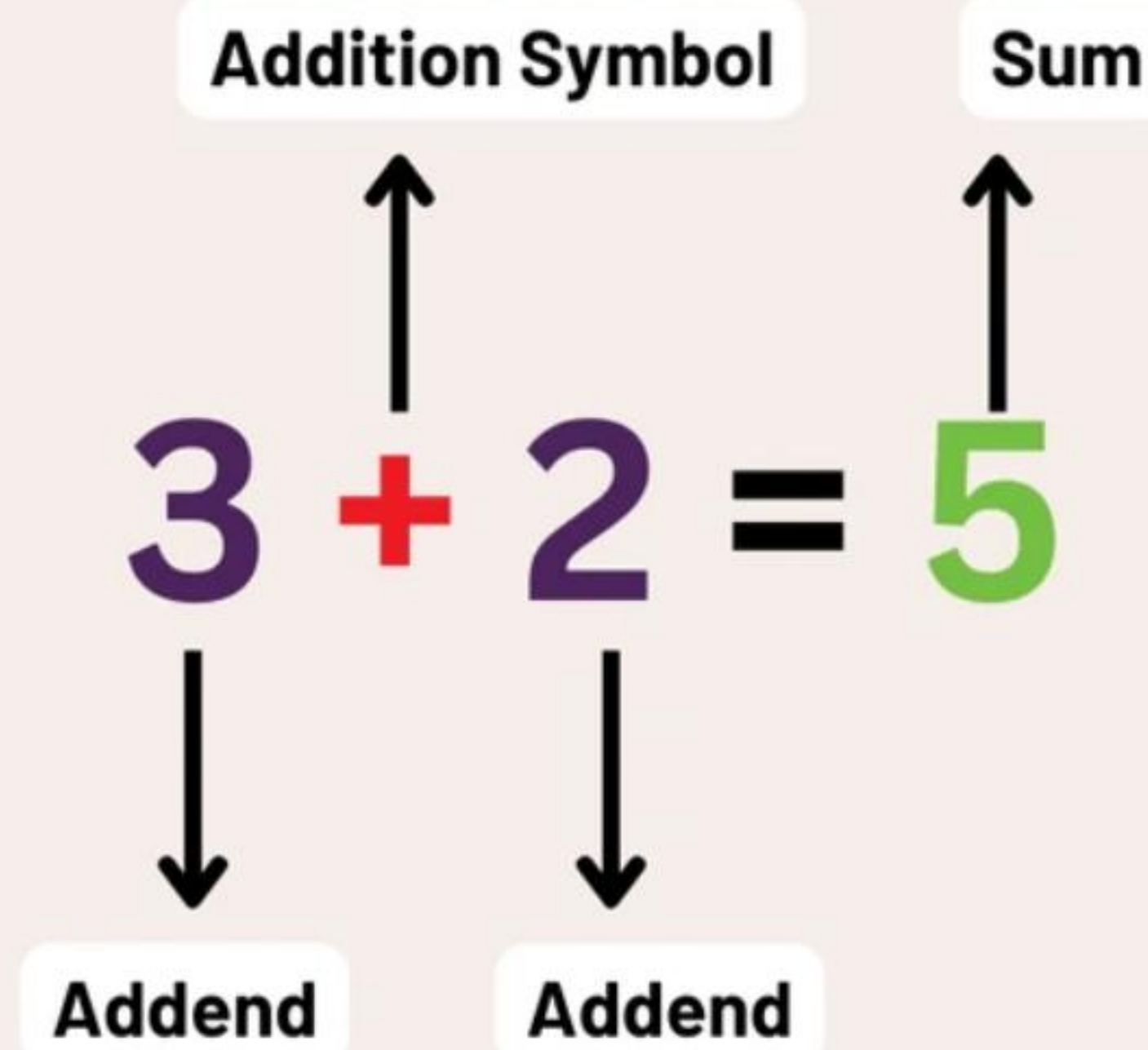
Establishing the foundational understanding of why and how Python handles special methods.

DEFINING OPERATOR OVERLOADING

Operator overloading allows the same operator to have **different meanings** depending on the context.

- 🔗 **Built-in Polymorphism:** The + operator adds numbers but concatenates strings.
- 🧩 **User-Defined Behavior:** Developers can define how operators interact with custom objects.
- ✂️ **Syntactic Sugar:** Makes code more readable and "Pythonic."

BASIC ADDITION



| THE BUSINESS CASE FOR OVERLOADING



Readability

`v1 + v2` is significantly more intuitive than `v1.add_vector(v2)`.



Integration

Enables custom objects to work seamlessly with Python's built-in functions like `sum()`.



Consistency

Standardizes interface behavior across diverse modules and libraries.

ANATOMY OF MAGIC METHODS

Magic methods, or **Dunder** (Double Underscore) methods, are special hooks invoked by the Python interpreter.

They are never called directly by the user (usually), but rather triggered by specific syntax like operators or built-in functions.

```
3
4 def showcase():
5     """Some code to showcase the syntax.
6
7     Docstrings are recognized and have an additional scope.
8     Color schemas can render them differently from other strings.
9
10    HACK doctests are highlighted too.
11
12    >>> print('''hello
13    ... world''')
14    """
15
16    @decorator(param='spam')
17    async def coroutine(db:aio_db.DatabaseConnection) -> List[str]:
18        r"""A coroutine."""
19
20        await logger.log('working\x12with %r', aio_db)
21
22        async with db.transaction():
23            print('Result: {result!r}'.format(result=
24                await db.query(...)))
25
26    mapping = None      # type: Dict[int, Any] # PEP 484
27
28    # a regular expression
29    get_regex = lambda: re.compile(      # type: ignore
30        r"""
31            word
32            (?
33                (?P<fill>.)?
34                (?P<align>[<=>^])      (?# another comment)
35            )?
36            another word\.\.\.
37            (?:\.(?P<precision>0|(?!\0)\d+))?
38        \Z""",
39        re.VERBOSE | re.DOTALL)
40
41    # NOTE Numbers with leading zeroes are invalid in Python3,
42    # use 0o...
43    answer = func(0xdeadbeef + 0b0010001 + 0123 + 0o123 +
44        # complex numbers
45        .10e12 + 2j) @ mat
46
47    return R'''No escapes '\\' in this \one'''
```


ARITHMETIC MASTERY: THE CORE TRIO

```
class Point:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Point(self.x + other.x,
                      self.y + other.y)

    def __sub__(self, other):
        return Point(self.x - other.x,
                      self.y - other.y)
```

+ `__add__`: Overloads the + operator.

- `__sub__`: Overloads the - operator.

x `__mul__`: Overloads the * operator.

Crucial: Always return a **new instance** to maintain immutability principles where appropriate.

ADVANCED ARITHMETIC OPERATORS

- $\div$ `__truediv__(self, other)`: Implements the floating-point division operator (/).
- $\%$ `__mod__(self, other)`: Implements the modulo operator (%), useful for circular logic.
- $\Downarrow$ `__floordiv__(self, other)`: Implements integer division (/ /).
- X^1 `__pow__(self, other)`: Implements the exponentiation operator (**).

COMPARISON DYNAMICS

06

Primary Methods

Comparison operators are essential for **Sorting** and **Equivalence** logic in complex systems.

`__eq__`: Equals (==)

`__ne__`: Not Equal (!=)

`__lt__`: Less Than (<)

`__gt__`: Greater Than (>)

OPERATOR TO METHOD MAPPING

Operator	Magic Method	Example Syntax
<=	<code>__le__(self, other)</code>	<code>a <= b</code>
>=	<code>__ge__(self, other)</code>	<code>a >= b</code>
&	<code>__and__(self, other)</code>	<code>a & b</code>

02. Advanced Hooks

Going beyond basic math to handle reflections, in-place updates, and object identity.

REFLECTED OPERATORS

What happens when the custom object is on the **right side**?
(e.g., `10 + my_obj`)

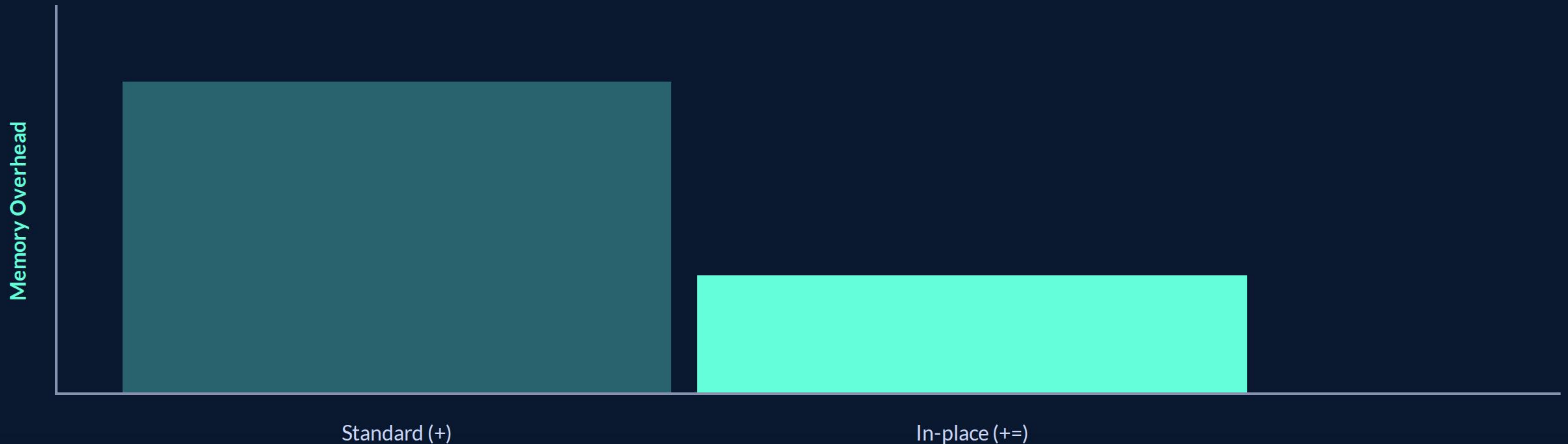
Python first calls the left operand's `__add__`. If that fails, it calls the right operand's `__radd__`.

```
def __radd__(self, other):  
    # Commutative property  
    return self.__add__(other)
```



IN-PLACE OPTIMIZATION

Augmented Assignment: Methods like `__iadd__` (`+=`) allow objects to be modified in-place, preventing unnecessary allocations.



STRING REPRESENTATIONS

`__str__`

Designed for the **End User**. Returns a readable, friendly version of the object.

```
print(car)
```

`__repr__`

Designed for the **Developer**. Should be unambiguous and look like Python code.

```
repr(car)
```


| BOOLEAN & LENGTH LOGIC

- ✔ **`__bool__(self)`**: Defines the truthiness of an object in `if` statements.
- 🔧 **`__len__(self)`**: Allows the `len()` function to return a value.
- ⚠ **Identity Rule**: If `__bool__` is missing, Python checks `__len__`. If both missing, it defaults to `True`.

| THE CONTAINER PROTOCOL

Transform your objects into sequences or mappings using these powerful hooks:

`__getitem__`: Access by index `obj[k]`.

`__setitem__`: Assignment `obj[k] = v`.

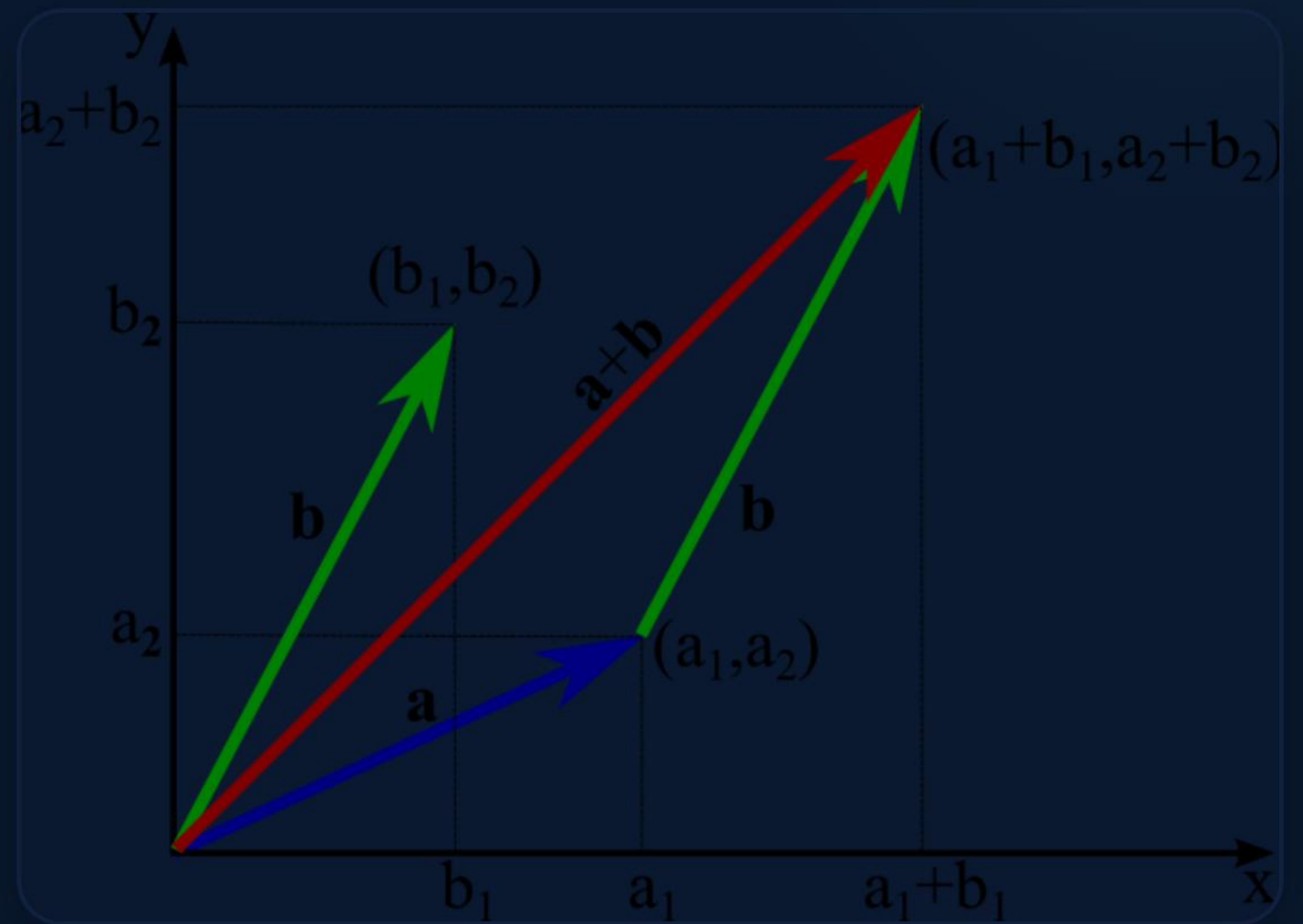
`__contains__`: Support for `in` operator.

```
def __getitem__(self, key):  
    return self.data[key]  
  
def __contains__(self, item):  
    return item in self.data
```


CASE STUDY I: VECTOR PHYSICS

Implementing vector addition and scaling using `__add__` and `__mul__`.

$$\vec{A} + \vec{B} = (x_a + x_b, y_a + y_b)$$



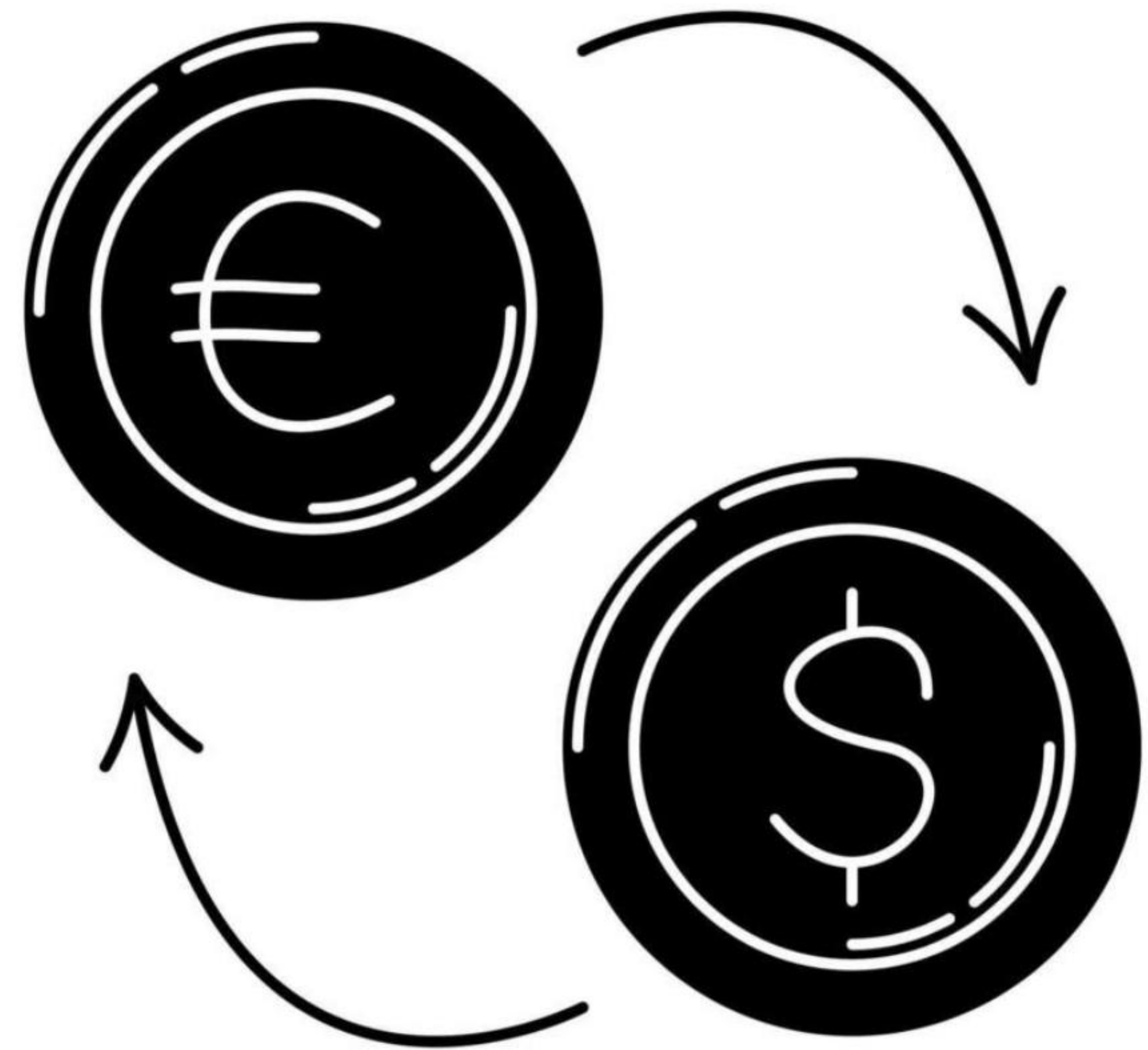
CASE STUDY II: CURRENCY SYSTEMS

Handling diverse currencies in a financial application requires precise operator behavior.

Validation: Ensure currencies match before adding.

Conversion: Overload `__mul__` for exchange rates.

Comparison: Compare wealth across different denominations.



BEYOND OPERATORS



`__call__`

Allows an instance to be called like a function: `obj()`.



`__getattr__`

Handles access to attributes that don't exist dynamically.



`__enter__`

Setup logic for the `with` context manager statement.

STRATEGIC GUIDELINES

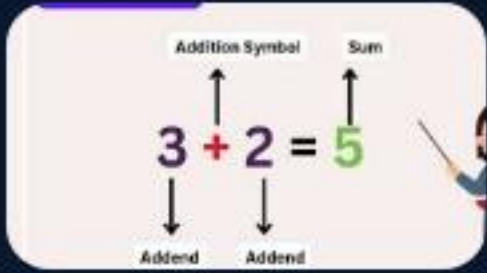
- 💡 **Be Intuitive:** Never overload + to perform subtraction. Follow mathematical standards.
- 🚫 **Avoid Over-engineering:** Don't overload every operator just because you can.
- 🤖 **Return NotImplemented:** Return this singleton if you don't support an operand type, allowing Python to try alternatives.

Thank You

Elevating Code Quality through Pythonic OOP Patterns

Questions & Discussion

IMAGE SOURCES



<https://eduseed.in/wp-content/uploads/2024/11/Addition.webp>

Source: eduseed.in



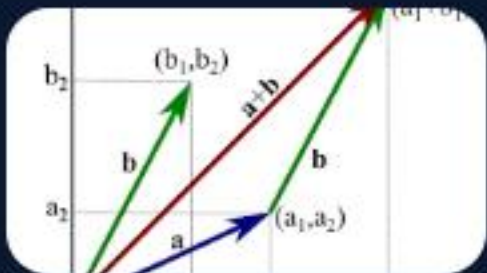
<https://camo.githubusercontent.com/9944acbb33bb3e448c1a8e693160d554c13b645910a61e974176374fe9fd7af3/68747470733a2f2f6d616769633737461636b2e6769746875622e69662f2f4d616769633507974686f6e2f6578616d706c652e706e67>

Source: github.com



<https://www.naturebasedtherapy.com.au/wp-content/uploads/2025/08/nature-as-a-mirror.png>

Source: www.naturebasedtherapy.com.au



https://mathinsight.org/media/image/image/vector_2d_add.png

Source: mathinsight.org



https://static.vecteezy.com/system/resources/previews/068/352/451/non_2x/currency-exchange-glyph-icon-euro-to-dollar-conversion-arrows-represents-trading-or-international-money-exchange-free-vector.jpg

Source: www.vecteezy.com